

实验五 在系统编程基础篇

一、实验目的

- ①熟悉数码管的两种驱动方法及其应用；
- ②掌握基本 verilog 语言的语法；
- ③学习使用 Quartus II 软件进行设计与仿真；
- ④学习在系统可编程器件的下载。

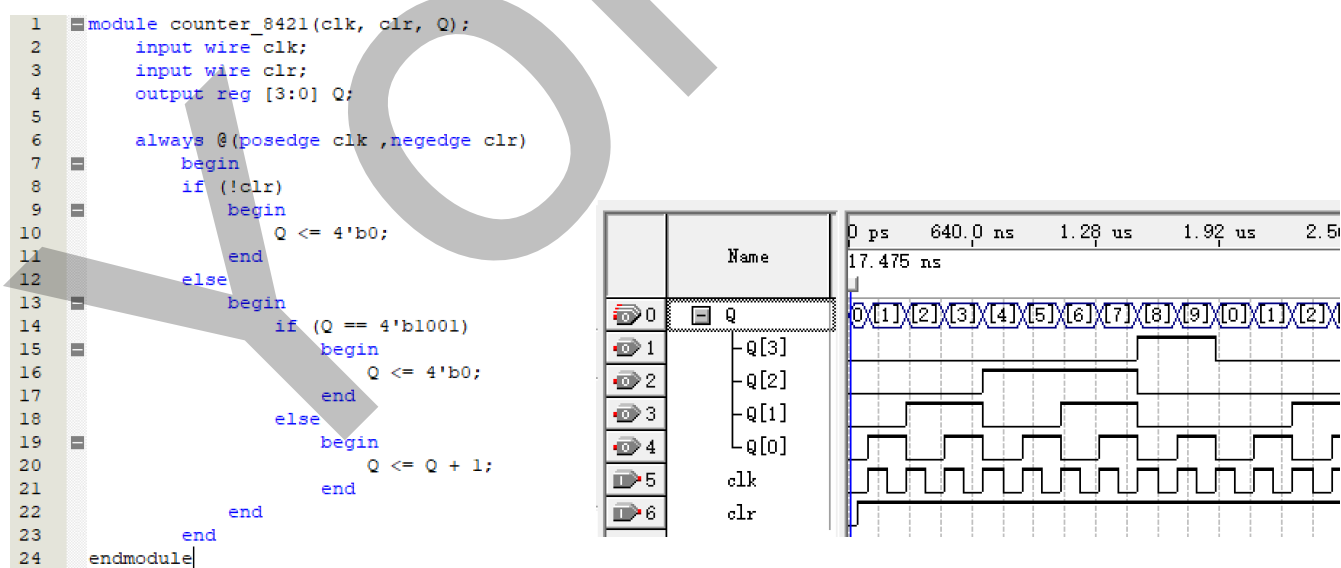
二、实验器件和设备

- TEC8 数字电路实验系统 1 台
- TBS1102B-EDU 双踪示波器 1 台
- QUARTUS II 软件

三、实验过程及结果

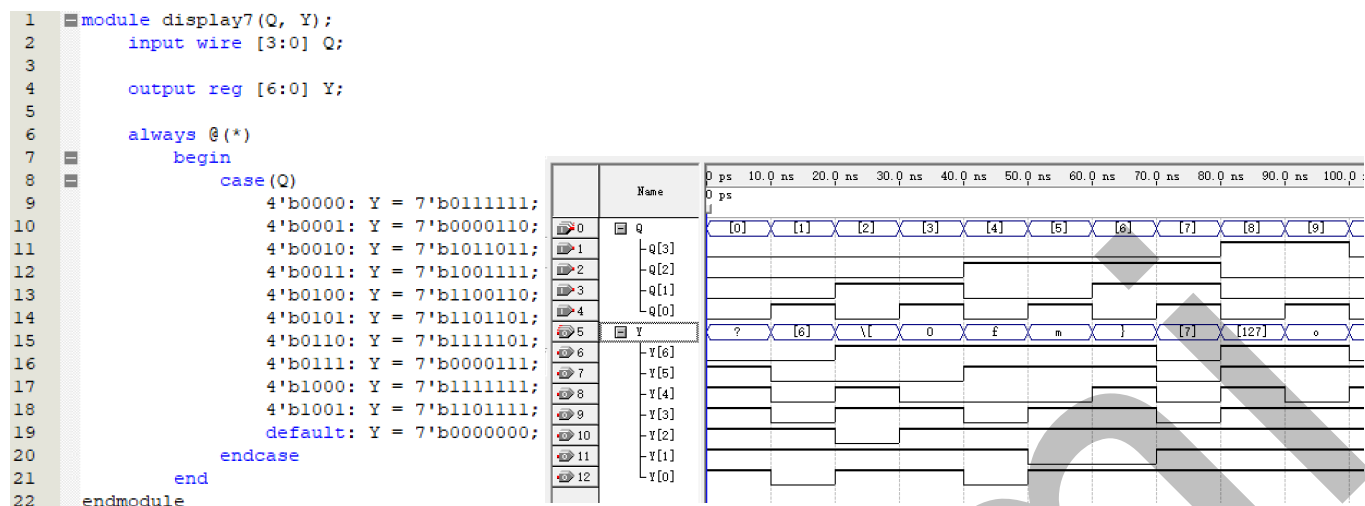
问题描述：用 verilog 语言设计并测试一个 8421 码十进制计数器及七段数码管显示系统。

子任务 2.1：用 QUARTUS II 软件设计异步复位 8421 码十进制计数器，对该计数器进行功能仿真；



代码和仿真截图如上，实现思路较为基础，不赘述。设计的 8421 码十进制计数器为上升沿有效、异步复位（复位端为低有效）。

子任务 2.2: 用 QUARTUS II 软件设计七段数码管显示, 对该数码管显示进行功能仿真;



代码和仿真截图如上, 编写用到 `case` 语句, 根据七段显示数码管的真值表进行译码, 仿真经检查无误 (出现乱码是由于视图问题以及七段显示数码管显示并不是十进制)。

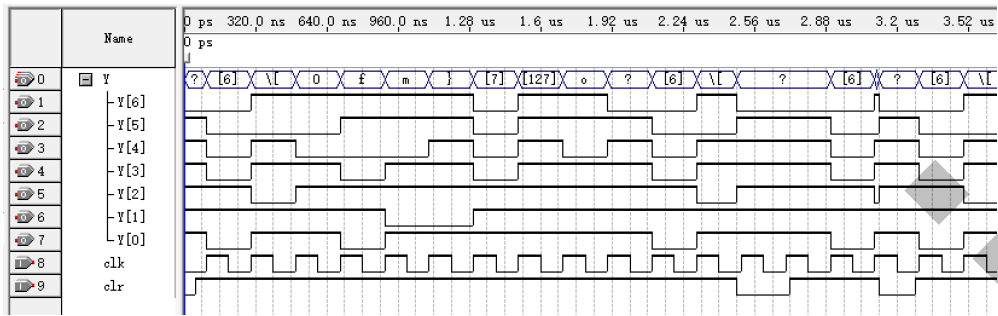
子任务 2.3: 用 QUARTUSII 软件实现十进制计数器并采用七段数码管显示, 对该系统进行功能仿真;

一、顶层文件代码:

```
1. module lab5(clk, clr, Y);
2.     input wire clk;
3.     input wire clr;
4.     output wire [6:0] Y;
5.
6.     wire [3:0] Q;
7.
8.     counter_8421 u1 (
9.         .clk(clk),
10.        .clr(clr),
11.        .Q(Q)
12.    );
13.
14.    display7 u2 (
15.        .Q(Q),
16.        .Y(Y)
17.    );
18.
19. endmodule
```

分别将基础模块 `counter_8421` 和 `display7` 实例化为 `u1` 和 `u2`, 综合得到的文件, 输入有时钟信号 `clk`、清零信号 `clr` 和 7 位输出 `Y`。

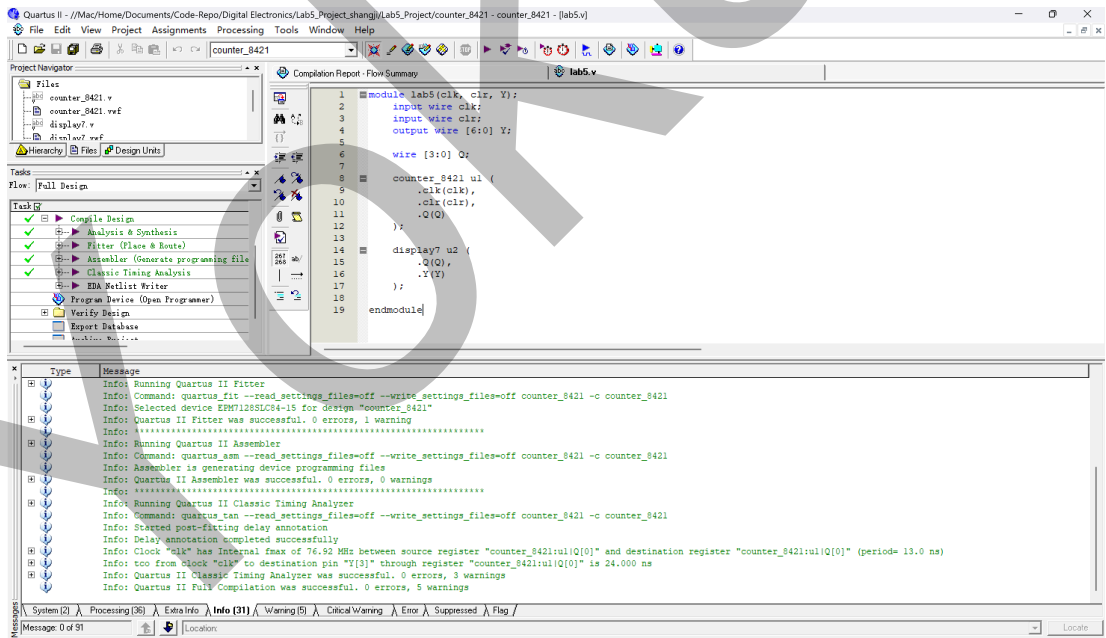
二、仿真结果：（前一段为计数器从 0 开始依次计数到最大的数码管的编码，后一段包含异步复位信号）



三、分配管脚图：

	Node Name	Direction	Location	I/O Bank	VREF Group	Reserved	Group	PCB layer
1	TCK	Input						
2	TDI	Input						
3	TDO	Output						
4	TMS	Input						
5	Y[6]	Output	PIN_51				Y[6..0]	
6	Y[5]	Output	PIN_50				Y[6..0]	
7	Y[4]	Output	PIN_49				Y[6..0]	
8	Y[3]	Output	PIN_48				Y[6..0]	
9	Y[2]	Output	PIN_46				Y[6..0]	
10	Y[1]	Output	PIN_45				Y[6..0]	
11	Y[0]	Output	PIN_44				Y[6..0]	
12	clk	Input	PIN_60					

四、综合编译结果：



四、实验总结

通过实验五，熟悉了 Verilog 编程语言语法以及 Quartus2 软件的设计与仿真，熟悉了分配引脚以及将程序烧录到硬件上的全流程。

教训是扁平电缆方向要插对且插紧！插紧！插紧！

实验六 在系统编程提高篇

一、实验目的

- ① 熟练掌握基本 verilog 语言的语法;
- ② 综合设计一个略复杂的数字电路;

二、实验器件和设备

- TEC8 数字电路实验系统 1 台
- QUARTUS II 软件

三、实验过程及结果

任务一：用 verilog 语言设计一个模 60 计数器

子任务 1.1：设计模 60 计数器

1. 源代码

```
1 module counter_60(clk, clr, method, Q, co);
2   input wire clk;
3   input wire clr;
4   input wire method;
5
6   output reg [5:0] Q;
7   output reg co;
8
9   always @(posedge clk, negedge clr)
10    begin
11      if (!clr)
12        begin
13          Q <= 6'd0;
14          co <= 1'b0;
15        end
16      else
17        begin
18          if (method)
19            begin
20              if (Q == 6'd59)
21                begin
22                  Q <= 6'd0;
23                  co <= 1'b1;
24                end
25              else
26                begin
27                  Q <= Q + 1;
28                  co <= 1'b0;
29                end
30            end
31          else
32            begin
33              if (Q == 6'd0)
34                begin
35                  Q <= 6'd59;
36                  co <= 1'b1;
37                end
38              else
39                begin
40                  Q <= Q - 1;
41                  co <= 1'b0;
42                end
43            end
44          end
45        end
46    end
47 endmodule
```

2. 引脚及功能分析:

- clk, 时钟信号 (clk, PIN_55);
- clr, 异步清 0 端 (clr, PIN_1);
- method, 模式选择 (正计时 or 倒计时均可实现!), (K0, PIN_54);

子任务 1.2：将输出传送给数码管

```
1 module display_7(Q, Y1, Y2);
2   input [5:0] Q;
3
4   output [3:0] Y1;
5
6   output [3:0] Y2;
7
8   assign Y1 = Q / 10;
9   assign Y2 = Q % 10;
10 endmodule
```

1. 源代码

2. 引脚及功能分析:

- Q, 输入, 接模 60 计数器的 6 位输出;
- Y1, 计数的十位, 输出给数码管 LG3 (PIN_35、PIN_36、PIN_17、PIN_18);
- Y2, 计数的个位, 输出给数码管 LG2 (PIN_37、PIN_39、PIN_40、PIN_41);

顶层设计:

1. 源代码

```
1 module lab6_counter60(clk, clr, method, Y1, Y2);
2     input wire clk;
3     input wire clr;
4     input wire method;
5     output wire [3:0] Y1;
6     output wire [3:0] Y2;
7
8     wire [5:0] Q;
9     wire co;
10
11     counter_60 u_counter_60 (
12         .clk(clk),
13         .clr(clr),
14         .method(method),
15         .Q(Q),
16         .co(co)
17     );
18
19     display_7 u_display_7 (
20         .Q(Q),
21         .Y1(Y1),
22         .Y2(Y2)
23     );
24
25 endmodule
```

2. 实现功能及设计思路总结:

- 实现了秒表（即模 60 计时器）的基础功能；
- 支持正计时和倒计时的切换；
- 设计思路，先设计模 60 计数器，再将计数传递给两个数码管，分别显示十位和个位（不需要译码，数码管的 4 个引脚分别对应四位二进制数）

任务二：用 verilog 语言设计一个电子琴；

子任务 2.1：按键对应

1. 源代码

```
1 module key_to_freq(key, m);
2     input [7:0] key;
3     output reg [15:0] m;
4
5     always @(*) begin
6         case (key)
7             8'b00000001: m = 16'd1908;
8             8'b00000010: m = 16'd1701;
9             8'b00000100: m = 16'd1515;
10            8'b00001000: m = 16'd1433;
11            8'b00010000: m = 16'd1276;
12            8'b00100000: m = 16'd1136;
13            8'b01000000: m = 16'd1012;
14            8'b10000000: m = 16'd956;
15            default: m = 16'd0;
16        endcase
17    end
18 endmodule
```

2. 引脚及功能如下:

- key, 对应 8 个电平开关输入 (K0~K7, PIN);
- m, 按键对应频率, 传递给模 m 计数器;

子任务 2.2：设计模 m 计数器

1. 源代码

```
1 module m_counter(clk, clr, m, out);
2     input clk;
3     input clr;
4     input [15:0] m;
5     output reg out;
6
7     reg [15:0] count;
8
9     always @(posedge clk, negedge clr) begin
10         if (!clr) begin
11             count <= 16'd0;
12             out <= 0;
13         end
14         else if (m != 16'd0) begin
15             if (count == m - 1) begin
16                 count <= 16'd0;
17                 out <= ~out;
18             end
19             else begin
20                 count <= count + 1;
21             end
22         end
23     end
24 endmodule
```

2. 引脚及功能说明:

- clk, 时钟信号, 接 1MHZ 时钟源 (PIN_55);
- clr, 异步清 0 信号 (PIN_1);
- m, 频率;
- out, 输出的分频, 传递给二分频;

子任务 2.3：二分频

1. 源代码

```
1 module divide_2(clk_in, clr, clk_out);
2     input clk_in;
3     input clr;
4     output reg clk_out;
5
6     always @(posedge clk_in, negedge clr) begin
7         if (!clr)
8             clk_out <= 0;
9         else
10            clk_out <= ~clk_out;
11     end
12 endmodule
```

2. 引脚及功能如下：

- clk_in, 来自计数器的信号；
- clr, 异步清 0 信号；
- clk_out, 输出波形, 传递给扬声器 (PIN_52);

顶层设计：

1. 源代码

```
1 module piano(clk, clr, key, speaker);
2     input clk;
3     input clr;
4     input [7:0] key;
5     output speaker;
6
7     wire [15:0] m;
8     wire m_clk;
9     wire speaker_clk;
10
11     key_to_freq key_to_freq_inst (
12         .key(key),
13         .m(m)
14     );
15
16     m_counter m_counter_inst (
17         .clk(clk),
18         .clr(clr),
19         .m(m),
20         .out(m_clk)
21     );
22
23     divide_2 divide_2_inst (
24         .clk_in(m_clk),
25         .clr(clr),
26         .clk_out(speaker_clk)
27     );
28
29     assign speaker = speaker_clk;
30 endmodule
```

2. 实现功能及设计思路：

- 电子琴的 8 个音弹奏；
- 设计思路, 由于驱动扬声器的必须是方波, 所以要输出音符, 则先 m 分频, 再 2 分频。不需要使用奇数分频。

任务三：用 verilog 语言设计一个寄存器堆；

子任务 3.1：设计 8 位寄存器 R0-R3, 负责存储数据

1. 源代码

```
1 module register_8(clk, clr, D, Q, en, CRW);
2     input clk;
3     input wire clr;
4     input wire en;
5     input wire CRW;
6     input wire[7:0] D;
7
8     output reg[7:0] Q;
9
10    always @(posedge clk, negedge clr)
11    begin
12        if (!clr)
13            Q <= 8'b0;
14        else if (!en && CRW)
15            Q <= D;
16        else
17            Q <= Q;
18    end
19 endmodule
```

2. 引脚及功能如下：

- clk, 时钟信号, 大家共用 (clk, PIN_55);
- clr, 异步清 0 端, 低电平有效 (clr, PIN_1);
- en, 使能信号, 来自 2-4 译码器, 任何一个时刻只有一个寄存器使能有效;
- CRW, 写信号, 共用 1 个, 来自电平开关 (K8, PIN_73);
- D, 数据, 来自电平开关 K0~K7;
- Q, 数据输出

子任务 3.2：设计 2-4 译码器，负责产生寄存器的使能信号

1. 源代码

```
1 module decode2_4(A, en_decode, Y);
2     input wire [1:0] A;
3     input wire en_decode;
4
5     output reg [3:0] Y;
6
7     always @(*)
8     begin
9         if (!en_decode)
10            begin
11                Y = 4'b1111;
12            end
13        else
14            begin
15                case(A)
16                    2'b00: Y = 4'b1110;
17                    2'b01: Y = 4'b1101;
18                    2'b10: Y = 4'b1011;
19                    2'b11: Y = 4'b0111;
20                    default: Y = 4'b1111;
21                endcase
22            end
23        end
24 endmodule
```

2. 引脚及功能如下：

- A，2 位地址（A_write），来自电平开关 K9~K10（PIN_70、PIN_69）；
- en_decode，使能信号，低电平有效（K13，PIN_65）；
- Y，4 位输出，给寄存器的使能信号（分别给 reg1~4）；

子任务 3.3：设计 4 选 1 选择器，负责选择一路数据输出

1. 源代码

```
1 module mux_4(A, D0, D1, D2, D3, Y);
2     input wire [1:0] A;
3     input wire [7:0] D0, D1, D2, D3;
4
5     output reg [7:0] Y;
6     always @(*) begin
7         case (A)
8             2'b00: Y = D0;
9             2'b01: Y = D1;
10            2'b10: Y = D2;
11            2'b11: Y = D3;
12            default: Y = 8'b0;
13        endcase
14    end
15 endmodule
```

2. 引脚及各功能如下：

- A，2 位地址（A_read），决定选择哪个寄存器（K11~K12，PIN_68、PIN_67）；
- D0、D1、D2、D3，4 个 8 位数据输入，来自四个寄存器；
- Y，8 位数据输出；

子任务 3.4：设计三态门，负责控制数据的输出

1. 源代码

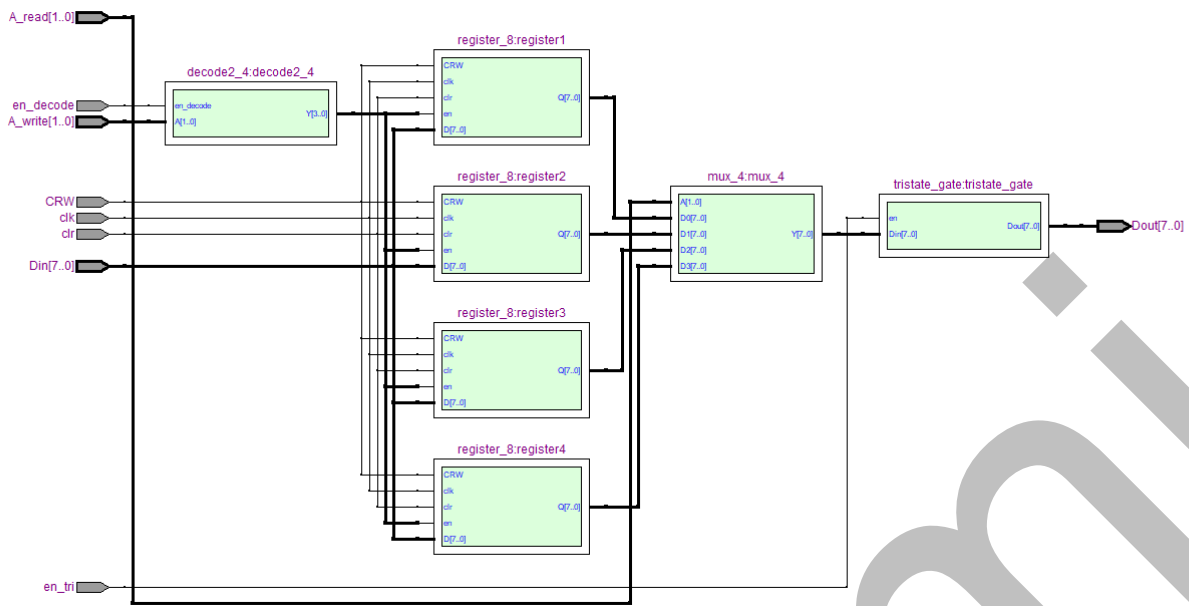
```
1 module tristate_gate(Din, en, Dout);
2     input wire [7:0] Din;
3     input wire en;
4
5     output wire [7:0] Dout;
6
7     assign Dout = !en ? Din : 8'bz;
8 endmodule
```

2. 引脚及各功能如下：

- Din，数据输入，来自数据选择器；
- Dout，数据输出，给 8 个 LED 灯；
- en，使能信号 en_tri，低电平有效，高电平时三态门高阻态（K14，PIN_64）。

顶层设计：

1. 网表文件（源代码较长）



2. 实现功能及设计思路:

- 寄存器堆的实现;
- 先设计基础模块, 再通过顶层文件整合, 将所有部件连接起来;